

Technical Summary Report: IPSec Architecture and Implementation

Executive Summary

Security work used to mean protecting a single machine from tampering or theft. Once networks connected those machines together, the job changed: the data itself had to be protected while it moved between systems, not just while it sat on a disk. Dr. Eric Affum frames this as protecting the information and the "ability to access information," rather than the hardware that holds it. IPSec extends IPv4 with the cryptographic operations needed to keep that information secure as it crosses networks that may include untrusted links.

Introduction and Operational Motivation

Organizations deploy IPSec because the alternative is real financial and intellectual loss. The 2007 CSI survey of computer security practitioners found that 46% of organizations had suffered a security incident that year, and the costliest incidents came from insider misuse of network access, human error, and deliberate espionage or trespass rather than outside attackers alone.

IPSec is built to counter two kinds of attack. Passive attacks - eavesdropping and traffic analysis - don't alter the data, which makes them hard to detect. The defense here is prevention through encryption. Active attacks, which include replay, masquerade, message modification, and denial of service, do alter the data stream or fabricate one. Preventing these outright is often impractical, so the goal shifts to detecting them and recovering afterward.

Within this framework, "internet security" means deterring, preventing, detecting, and correcting violations that occur while information is in transit across interconnected networks.

Core Security Services

IPSec's security services follow the X.800 (OSI Security Architecture) model, built around confidentiality, integrity, and availability, with authentication and access control added on top:

Confidentiality protects transmitted data against passive attacks and covers traffic flow analysis as well as message content.

Authentication has two forms. Peer entity authentication confirms the identity of each party when a connection is established, which guards against masquerade and replay. Data origin authentication confirms who sent a given data unit - useful for something like email, where the parties haven't already established a session - but it doesn't stop someone from duplicating that unit later.

Integrity also splits into two types. Connection-oriented integrity guards against message stream modification and denial of service by catching duplication, insertion, and reordering. Connectionless integrity protects individual messages from modification without reference to any ongoing session.

Access control limits and shapes what an authenticated entity can reach on a host or through an application.

Availability is the property that a resource stays accessible and usable on demand by an authorized party. It is the service that defends specifically against interruption attacks.

Protocol Specification: AH vs. ESP

IPSec relies on two protocols, and each maps onto a different set of security mechanisms.

The **Authentication Header (AH)** handles data integrity and the authentication exchange. It depends on a digital signature - a cryptographic transformation applied to a data unit that proves where it came from and that it hasn't been altered. AH proves the source of a packet, but it has no encipherment mechanism, so it offers no confidentiality at all.

Encapsulating Security Payload (ESP) goes further. It provides confidentiality through encipherment, running the data through an algorithm that makes it unreadable to anyone without the key. It also applies traffic padding - inserting extra bits into the gaps of a data stream - specifically to make traffic analysis harder. ESP can additionally provide integrity and authentication, which means it covers most of what AH does on its own.

Deployment Modes: Transport vs. Tunnel

IPSec runs in one of two modes, consistent with the network access security model.

Transport mode sets up a direct logical channel between two endpoints and applies the security transformation to the upper-layer data only.

Tunnel mode works more like a gatekeeper. It takes the entire original IP packet, headers included, and treats that whole packet as the plaintext input for a new transformation. Because the original headers get wrapped rather than exposed, tunnel mode also shields a system's internal routing information from an outside attacker, whether that's a person or malicious software.

Security Association (SA) Architecture

A Security Association is the "secret information" piece of the network security model. It's the formal agreement between two parties that establishes a shared secret and makes the security transformation possible between them.

Setting up an SA involves three things. First, the algorithm has to be defined - AES or SHA, for instance - chosen so that an attacker can't defeat it. Second, the keys used with that algorithm need to be generated and distributed. Third, the specific protocol, AH or ESP, has to be specified, since that's what actually applies the algorithm and the key material to produce the security service.

Key Management Framework

Distributing and managing cryptographic keys is its own problem, and IPSec handles it two ways.

Manual management means physically distributing the secret information, or relying on a trusted third party to resolve disputes about who a key really belongs to. It doesn't scale well, but it's simple and doesn't depend on any online infrastructure.

Automated management, through ISAKMP/Oakley, uses a multi-step exchange - typically seven messages - with a public-key authority that automates fetching public keys and confirming they're still current.

The table below lays out how symmetric and public-key encryption differ, since both show up in key management:

Feature	Symmetric (Conventional) Encryption	Public-Key (Asymmetric) Encryption
Key count	One key, shared by both parties	Two related keys: public and private
Security basis	Secrecy of the key itself	Difficulty of factoring large numbers (e.g., RSA)
Mathematical basis	Substitution and transposition	One-way and trap-door functions
Main challenge	Getting the single key distributed securely	Making it computationally infeasible to derive the private key from the public one

Public-Key Certificates

Running a central authority online around the clock is a bottleneck, so IPSec implementations use public-key certificates instead. A certificate ties an identity to a public key and is signed by a certificate authority (CA).

For a participant A, that certificate is written as:

$$CA = E(PR_{auth}, [T \parallel ID_A \parallel PU_A])$$

where PR_{auth} is the CA's private key and T is a timestamp.

A working certificate scheme needs to satisfy four conditions. Anyone should be able to read the certificate and pull out the name and public key it contains. Anyone should be able to confirm it actually came from the CA. Only the CA should be able to issue or update certificates. And anyone should be able to check the timestamp T to confirm the certificate is still current, which stops an old, compromised key from being replayed.